Benchmarking AI Factories on MeluXina Supercomputer

Challenge Evaluation Report

EUMaster4HPC Student Challenge 2025/2026

Team 3

Tommaso Crippa
Edoardo Leali
Emanuele Caruso

February 2026

1 Project Summary and Deliverables

1.1 Challenge Overview

The EUMaster4HPC Student Challenge 2025 presented us with a clear objective: creating a benchmarking solution for AI factories running on supercomputer infrastructure. As organizations increasingly deploy large language models in production environments, the need for systematic performance evaluation on HPC systems has become critical. Our team tackled this problem by building a comprehensive toolkit tailored to the MeluXina supercomputer.

Working with HPC infrastructure introduces unique constraints not found in typical cloud deployments. The SLURM scheduler dynamically assigns compute nodes to jobs, meaning service locations are unknown until runtime. Containers must run through Apptainer. GPU resources are shared among many users with strict allocation policies. Our framework was designed from the ground up to operate within these realities.

1.2 Framework Architecture

We developed a modular toolkit comprising three interconnected components, each addressing a distinct aspect of the benchmarking workflow:

- **Server Module:** Manages the lifecycle of AI inference services on the cluster. When users request a vLLM server deployment, this component generates SLURM batch scripts, submits jobs, monitors allocation status, and registers the assigned endpoint for other components to discover.
- **Client Module:** Drives workload generation against deployed services. Two operational modes are supported: a user-simulation mode where virtual clients make requests with configurable delays between them, and a saturation mode that pushes requests at maximum rate to find throughput limits. Performance data including response times and success rates are captured automatically.
- **Monitor Module:** Orchestrates Prometheus deployments to scrape runtime metrics from inference servers. Target endpoints are resolved automatically through our discovery system, and collected data can be exported for offline analysis in standard formats.

A key architectural decision was implementing a file-based service registry. When any server starts, it writes its network coordinates to a shared location. Benchmarking and monitoring components consult this registry to find their targets, removing the need for manual endpoint copying between jobs.

Configuration follows a declarative model using YAML files. Each recipe captures everything needed for a deployment or benchmark: resource requests, runtime parameters, workload characteristics. Storing recipes in version control makes experiments repeatable and shareable.

1.3 Evaluation Criteria

We evaluated our work against four criteria:

Working end-to-end pipeline. The complete workflow from deploying the first server, to collecting the final results executes successfully on MeluXina, with minimal manual intervention. Each component functions correctly in isolation and integrates properly with the others through the discovery mechanism.

Maintainable codebase. Consistent patterns across modules (Manager classes for logic, Orchestrator classes for SLURM interaction) make the code understandable for new developers. Adding support for new inference backends requires implementing well-defined interfaces without modifying core logic.

Meaningful performance data. Benchmarks produce actionable metrics: latency distributions, throughput figures, error counts. The monitoring stack captures vLLM-specific telemetry including token generation rates and memory utilization. Output formats (JSON, CSV) enable further analysis with standard tools.

We validated these properties through extensive testing: four distinct benchmark scenarios totaling over 7,200 inference requests, all completing without errors across simple, stress, throughput-focused, and memory-intensive workloads.

2 Team Organisation

2.1 Navigating Early Setbacks

Our team's path through this challenge was unconventional. Just a few weeks after the project began, two of the original four members departed from the EUMaster4HPC program entirely. This left the remaining members in a difficult position during the opening phases, when foundational decisions about architecture and implementation approach needed to be made.

This is why during the first phase, progress was slow. The reduced team managed basic preparatory work (setting up the repository and gaining familiarity with MeluXina's environment) but lacked the capacity to push into serious development.

Towards the end of October Tommaso Crippa joined the group. With his addition, the team started making some real progress. The subsequent two months, from late November through January 2026, saw intensive development to catch up to other teams.

In the final month we focused on running comprehensive benchmark suites on MeluXina, writing documentation, and preparing materials for submission. What we achieved in roughly eight weeks of concentrated effort compensated substantially for the slow start.

2.2 Who Did What

Tommaso Crippa:

- Project coordinator and primary point of contact for challenge communications
- Implementation of Server Module
- Integration of other modules in final framework
- Execution of benchmarks and results analysis

- Wrote documentation and presentation

Edoardo Leali:

- Developed Client Module
- Implemented Workload generation logic
- Handled initial project setup

Emanuele Caruso:

- Created Monitor Module prototype

2.3 How We Worked Together

The coordination was very difficult since each team member is pursuing studies at a different university, which meant class schedules and time zones frequently didn't align. Rather than struggling to synchronize for regular meetings, we shifted toward asynchronous workflows. Our communication happened through messaging platforms, where individuals could update their progress, ask questions, and share blockers without requiring everyone to be present simultaneously.

We used GitHub to host our codebase. Since our project was very modular we could make progress in parallel without constant synchronization overhead.

The strategy worked because it matched our constraints. What we lost in real-time collaboration, we gained in flexibility and the ability to maintain momentum despite being spread across different institutions.

3 Lessons and Takeaways

3.1 New Technical Capabilities

- **System design:** Creating three modules that interoperate cleanly demanded thoughtful architecture. Separating concerns between managers and orchestrators, defining stable interfaces, making configuration declarative, these patterns will serve us in future projects.
- **LLM serving infrastructure:** Deploying vLLM taught us how inference servers actually behave: how latency grows with input length, how batching affects throughput, what metrics matter for production systems. This goes well beyond running models in notebooks.
- **Metrics pipelines:** Building the monitoring component required understanding Prometheus. We now know how to instrument services and build observability into systems from the start.

3.2 Personal Growth

- **Resilience under pressure:** Starting with half a team taught us to pivot and adapt. We learned to focus on achievable milestones and build momentum grad-

ually, rather than trying to achieve an impossible goal without the necessary resources.

- **Distributed teamwork:** Coordinating across locations and schedules within different countries network meant relying heavily on asynchronous communication and clear task boundaries. We got better at making progress without requiring everyone online simultaneously.